

Neural Network Introduction

By Dr Shantanu Pathak

Neural Network

- Network of neurons with multiple layers
- Types of Neural Network
- Supervised Learning:
 - Neural Network (NN)/ Simple Neural Network (SNN) / Neural Network with Back-propagation / Feed-forward neural Network (FFNN)
 - CNN Convolutional Neural Network
 - RNN Recurrent Neural Network
 - LSTM Long Short Term Memory Neural Network
 - Transformer
- Unsupervised Learning
 - Auto-Encoders
- Both
 - GPT Generative Pre-trained Transformers
 - GAN Generative Adversarial Neural Networks

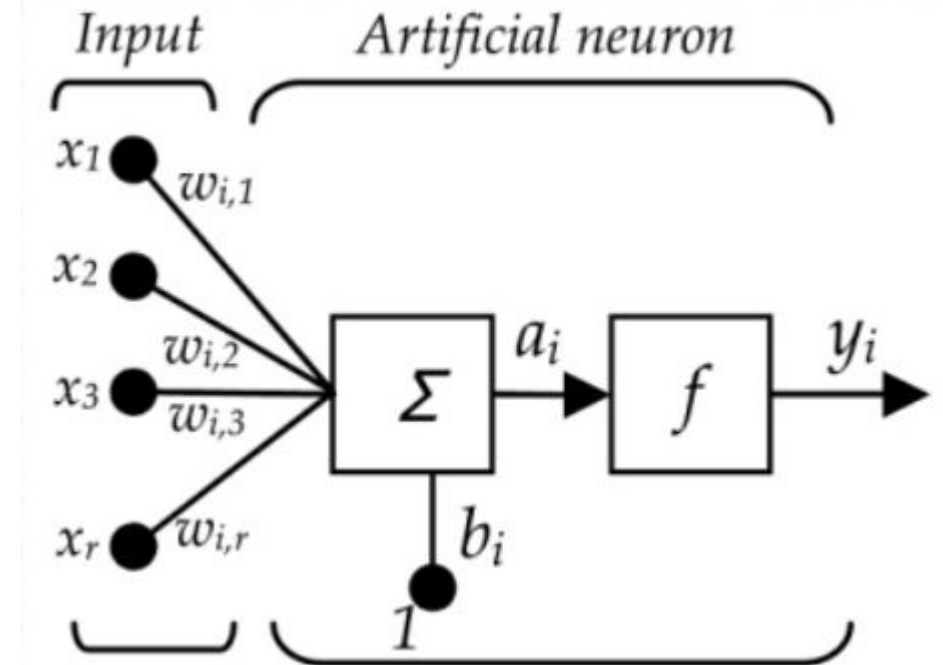
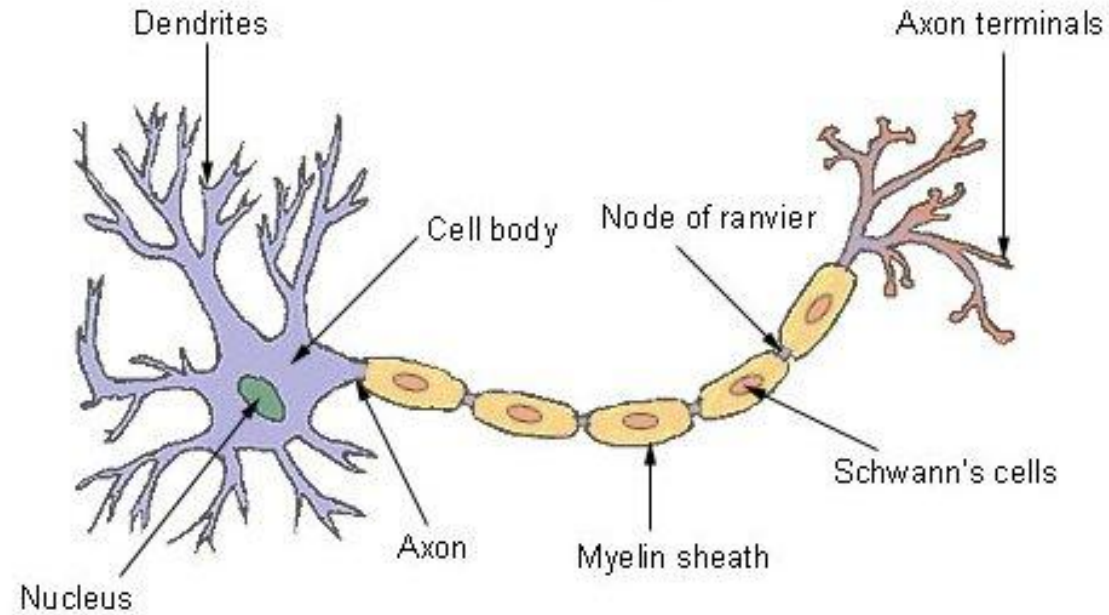
Single Neuron Architecture

- This is basic unit of processing in a neural network
- One neuron can have many inputs
- It gives ONLY one output
- For every input there is weight associated with it
- It has two parts Summation and Activation function

- Summation formula is

$$X = \sum_{i=1}^n (w_i * x_i) + b$$

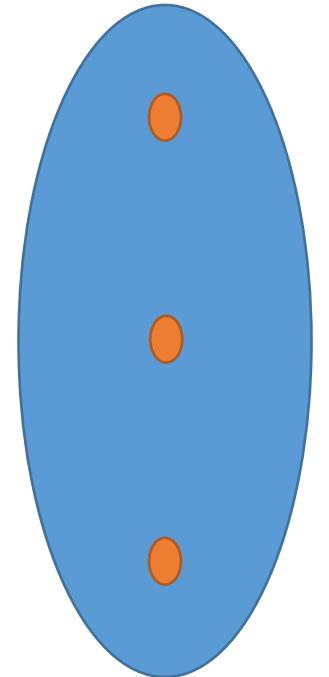
- Activation Function takes output of summation and applies a given function on it. This generate the output of neuron (y)
- Another name for a single neuron is **perceptron**



A Single Layer of Neural Network

- Single layer contains multiple neurons together
- Every neuron gets some inputs and generates 1 output
- All neurons in single layer have same activation function
- Neurons in same layer are not connected to each other
- There are various types of layers
- Ex. Dense layer, convolutional layer, RNN layer, etc
- NOTE: Neural Network with single layer is also known as SVM (Support Vector Machine)

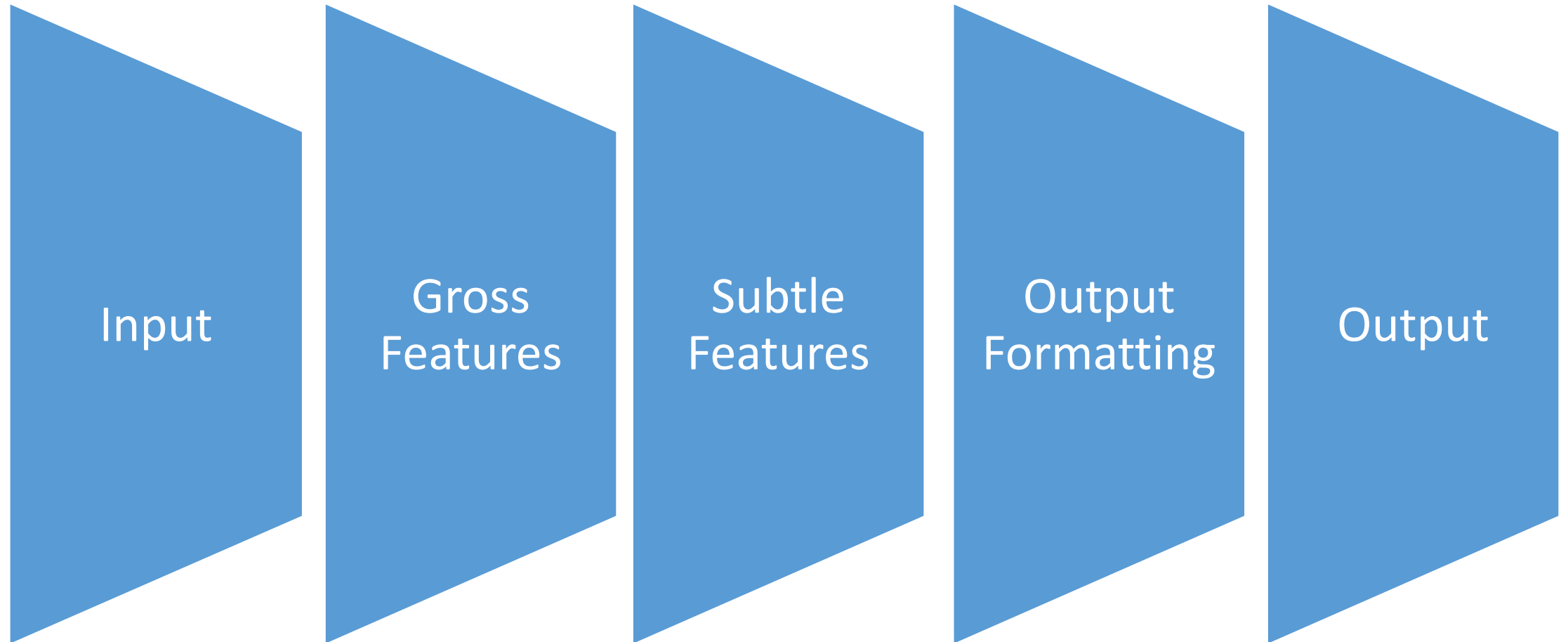
Ex. Single layer with
3 neurons



Multiple Layer Neural Network

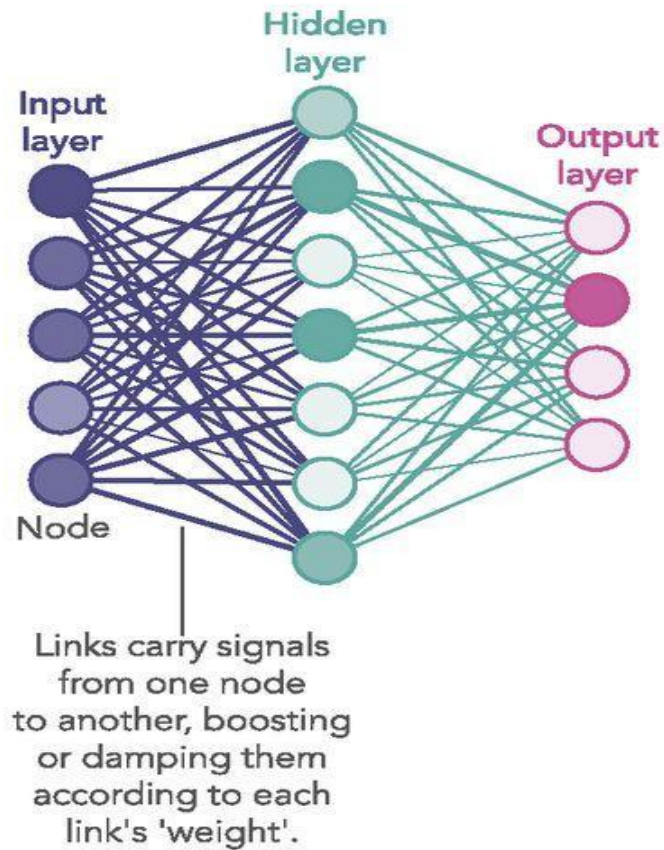
- Neural Network with multiple layers is multi-layer neural network
- Here first layer is input layer and last layer is output layer
- All layers between input and output layers are called as hidden layers, because they are not exposed to the user
- Another name to such neural network is MLP (Multi Layer Perceptron)

Intuition of Multi Layer NN Architecture

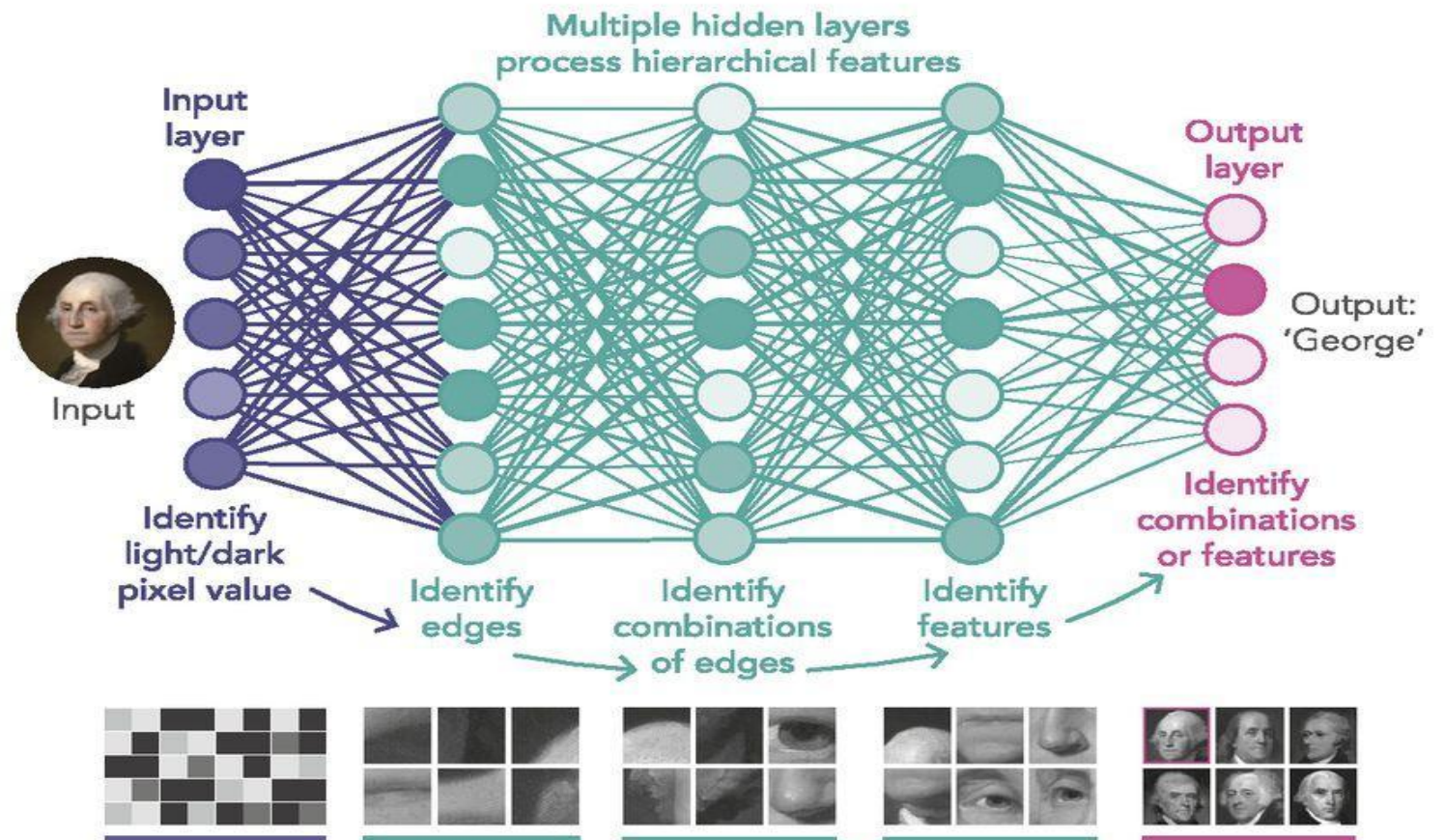


Intuition of Multi Layer NN Architecture

1980S-ERA NEURAL NETWORK



DEEP LEARNING NEURAL NETWORK



Hyper Parameters for NN

- Number of Layers
- Number of Neurons in Each Layer
 - Input Layer
 - Output Layer
 - Hidden Layer
- Activation Function
- Learning Rate
- Batch Size
- Epochs
- Optimization Algorithms
- Loss Functions
- Regularization
- Weight Initialization

Activation Functions

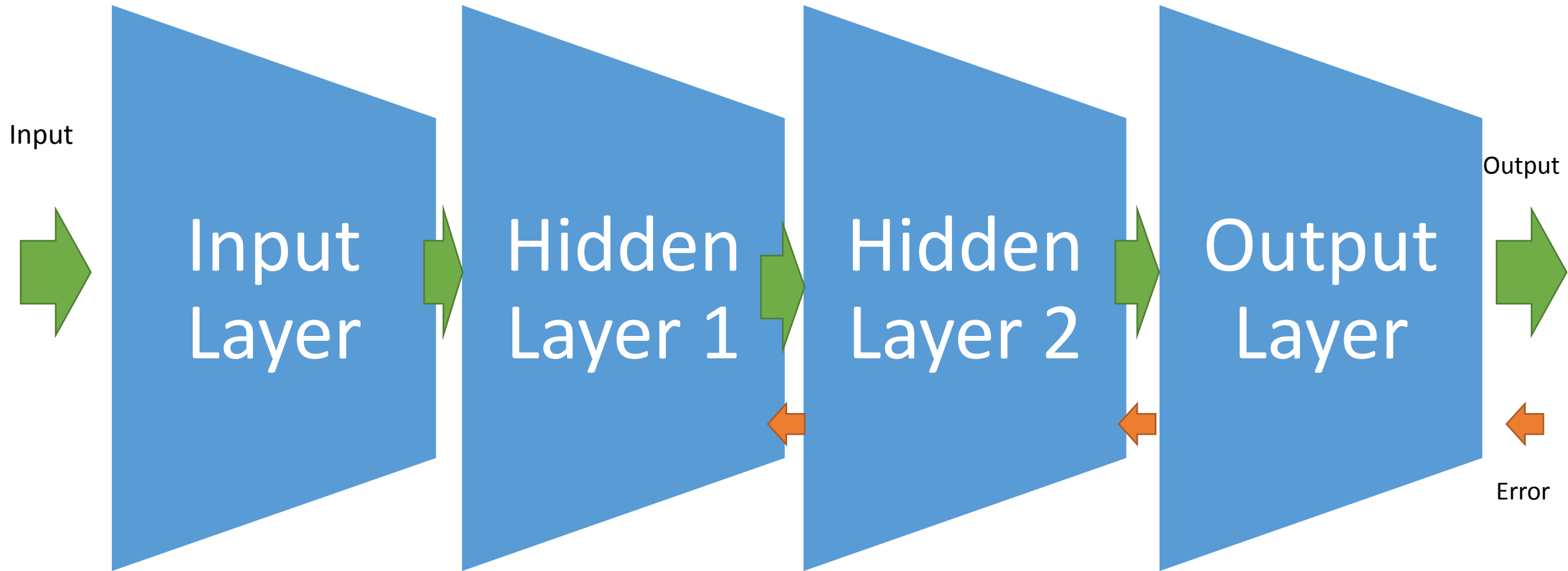
- Linear $f(x) = x$ so $y = x$
- Sigmoid
- TanH (Hyperbolic Tangent)
- ReLU (Rectified Linear Unit)
- GELU (Gaussian Error Linear Unit)
- Softmax

Determining the Right Activation Function for Hidden Layer

- Problem
 - Linearly separable data
 - use linear activation (linear)
 - Non-linearly separable
 - use non-linear activations (ReLU / GELU / Sigmoid)
- Output
 - Between 0-1 (Sigmoid)
 - Having Negative Values (GELU / TanH)
- Speed of convergence
 - RELU is fast to converge
- Vanishing gradient problem
 - Use non-saturating activation like ReLU / GELU / linear
 - Avoid use of saturating activation like sigmoid or tanh
 - Use proper weight initialization

Process of Training in Neural Networks

Using Back propagation of Error

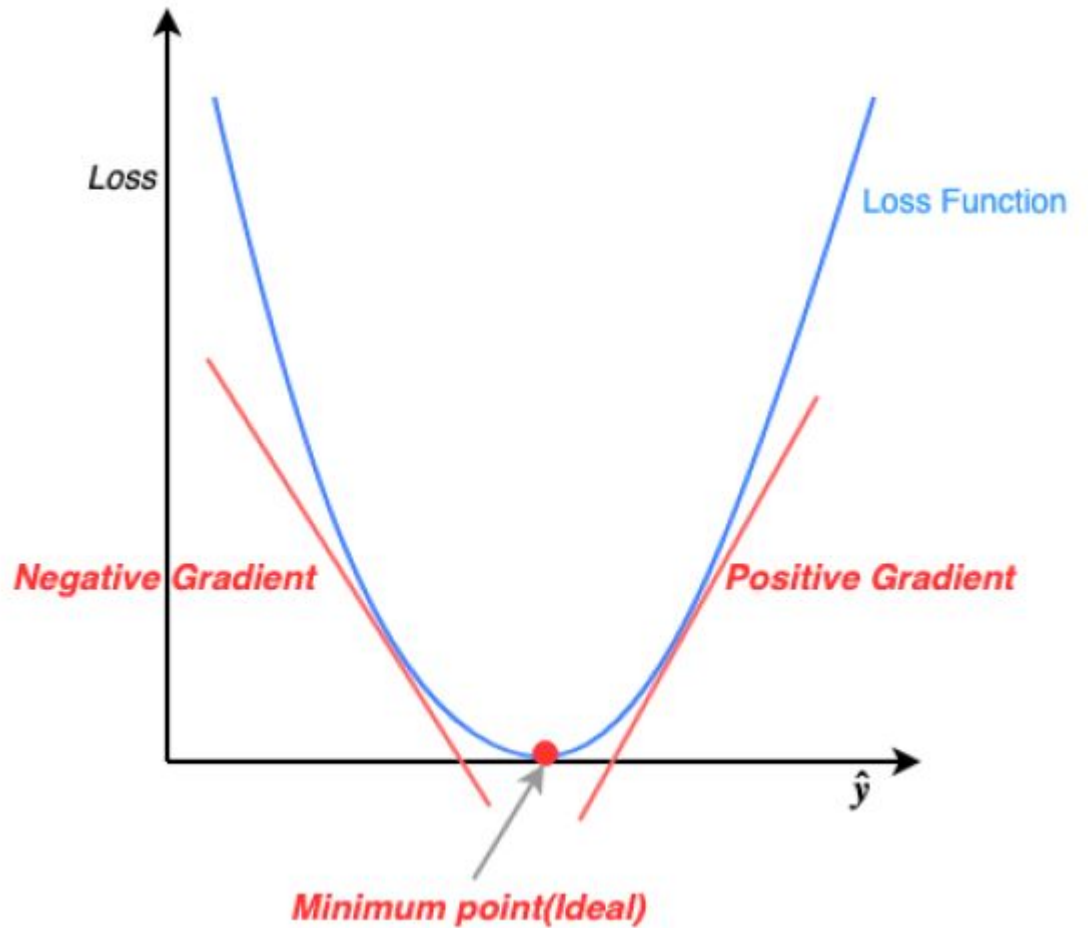


Weight Update in Back Propagation

- $W = W - lr * \text{gradient}$
- where
 - lr is learning rate
 - gradient = derivative of loss w.r.t weight
 - This is rule for weight update in gradient descent (also known as delta rule)

Derivative of Loss / Error in Weight Update

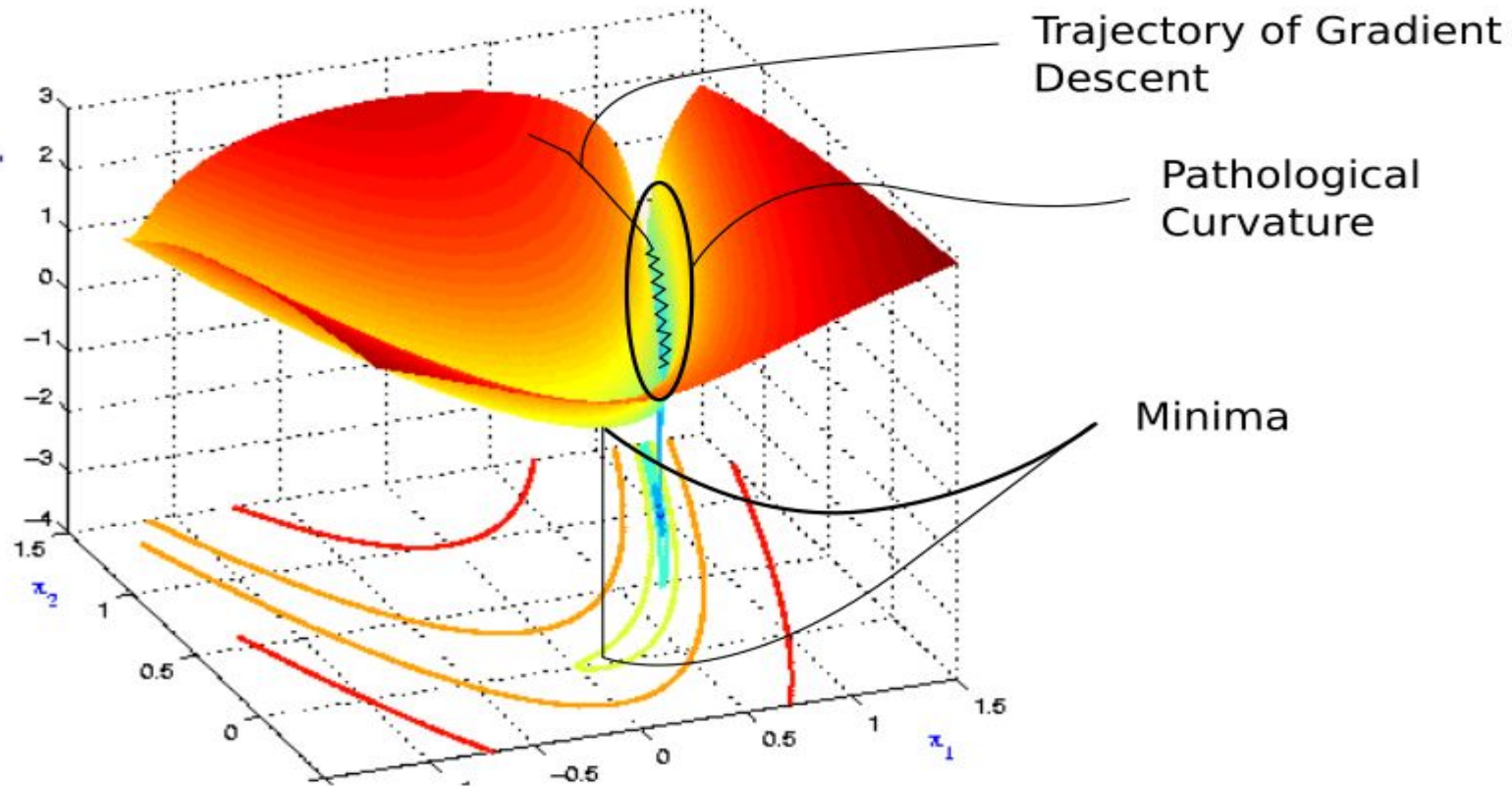
- The **derivative** of the loss function dictates whether to increase or decrease weights.
- A positive derivative would mean decrease the weights and negative would mean increase the weights.
- The steeper the slope the more incorrect the prediction was.



Optimizers in NN

- **Stochastic Gradient Descent (SGD)** (Update weight as per gradient)
 - Weight update rule of gradient descent is also known as delta rule
- **Momentum** (Weighted Average of Gradient)
- **RMS prop** (Exponential Average of gradients)
- **Adam**
 - momentum accelerates our search in direction of minima, RMSProp impedes our search in direction of oscillations.
 - Adam or Adaptive Moment Optimization algorithms combines the heuristics of both Momentum and RMSProp.

Look of sample 3D Loss Curve



- In reality there are n inputs so loss curve is n -Dim!